

9

Producing forms

This chapter demonstrates how to use PHP scripts to process HTML form data.

Performing actions

Checking set values

Validating form data

Filtering data

Sending hidden data

Handling submissions

Making sticky forms

Uploading files

Surrounding forms

Appending link data

Summary

Performing actions

Processing data submitted from an HTML form on a web page is perhaps the most fundamental task performed by a PHP script.



The HTML **<form>** tag's **action** attribute is used to nominate the PHP script that will process ("handle") the form data. Additionally, the **<form>** tag's **method** attribute is used to specify how the data is sent to the script – typically by the **POST** method. So the HTML form tag and attributes might look like this:

```
<form action="script.php" method="POST">
```

PHP provides a special "superglobal" **\$_POST** array variable in which it stores form data submitted by the **POST** method. It creates an array element of the same name as each submitted form element name, storing the value they contain. For example, with this form element:

```
<input type="text" name="email" >
```

On submission of the form, PHP will create an array element named **\$_POST['email']** containing the value entered into that form input by the user.



This is case-sensitive so the capitalization must be precisely as described – **\$_POST['Email']** or **\$_Post['email']** are incorrect, for example.

For easy recognition, PHP variables of the same names as the form elements can be assigned the values stored in their corresponding **\$_POST** array elements. The values can then be displayed in a response page created by the action handler script:



Create an HTML document containing a form with three text input fields for data submission

```
<form action="action_handler.php" method="POST">
```

```

<dl>
<dt>Name:
<dd><input type="text" name="name">
<dt>Email Address:
<dd><input type="text" name="mail">
<dt>Comments:
<dd><textarea rows="5" cols="20" name="comment" >
</textarea>
</dl>
<p><input type="submit" ></p>
</form>

```



action.php

Now, save the document in your web server's **/htdocs** directory as **action.php**

Next, create an HTML document containing a script to assign form values to like-named PHP variables for output

```

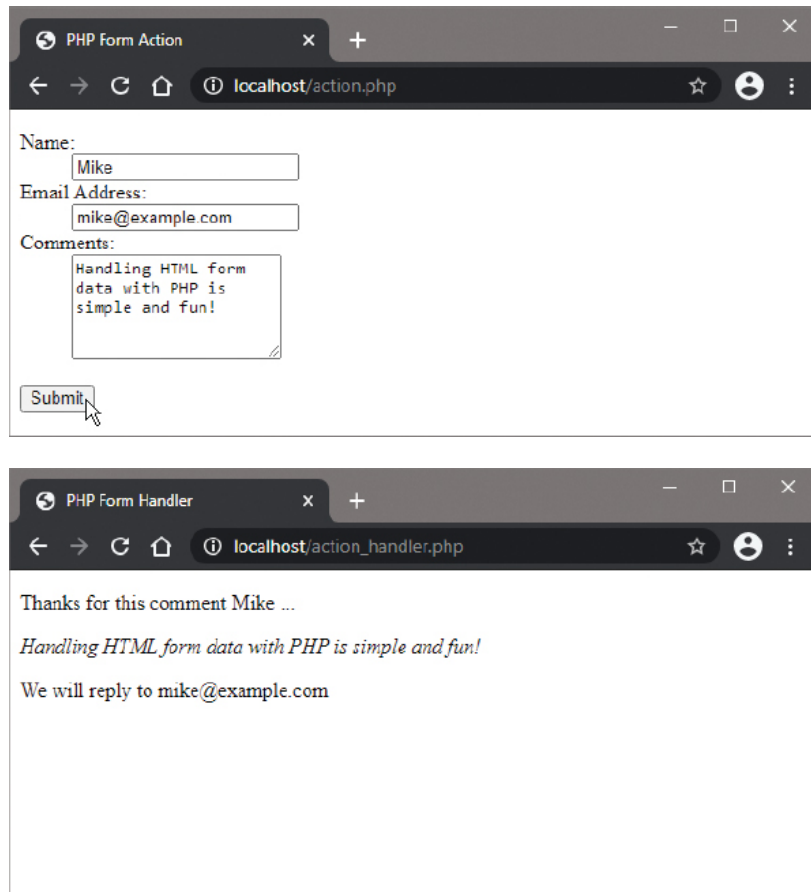
<?php
$name = $_POST['name'] ;
$mail = $_POST['mail'] ;
$comment = $_POST['comment'] ;
echo "<p>Thanks for this comment $name ...</p>" ;
echo "<p><i>$comment</i></p>" ;
echo "<p>We will reply to $mail</p>" ;
?>

```



action_handler.php

4 Save the document in your web server's **/htdocs** directory as **action_handler.php** then open **action.php** via HTTP and submit some data to see it get displayed in the response page produced by PHP script



The first screenshot shows a web browser window titled 'PHP Form Action' with the address bar displaying 'localhost/action.php'. The page contains a form with three input fields: 'Name:' with the value 'Mike', 'Email Address:' with the value 'mike@example.com', and 'Comments:' with the text 'Handling HTML form data with PHP is simple and fun!'. A 'Submit' button is at the bottom, with a mouse cursor clicking it.

The second screenshot shows a web browser window titled 'PHP Form Handler' with the address bar displaying 'localhost/action_handler.php'. The page displays the submitted data: 'Thanks for this comment Mike ...', 'Handling HTML form data with PHP is simple and fun!', and 'We will reply to mike@example.com'.



Form submission methods may be either “GET” or “POST”. The **GET** method appends name=value pairs to the URL but is mostly used to request information – such as retrieving a database record. The **POST** method can submit more data, is more secure as the data is not appended to the URL, and is mostly used when an action is to be performed – such as updating a database record. The **POST** method is therefore preferred for most examples in this book.

Checking set values

PHP can easily ensure that the user has entered data into submitted HTML form fields using the built-in **isset()** function. This function takes the name of a variable as its argument and returns **TRUE** only if that variable value is not **NULL** – thus ensuring that the variable has been “set” with some value. Empty form fields will be **NULL** unless the user has entered a value.



Conditional tests can be made with the **isset()** function to indicate where form fields have been left empty, or to provide a response page where the fields have been completed:

- 1 Create an HTML document containing a form with three radio buttons providing a single field for data submission

```
<form action="isset_handler.php" method="POST">
<fieldset>
<legend>What kind of language is PHP?</legend>
Scripting<input type="radio"
        name="definition" value="Scripting"> <br>
Markup<input type="radio"
        name="definition" value="Markup"> <br>
Programming<input type="radio"
        name="definition" value="Programming">
</fieldset><p><input type="submit" ></p>
</form>
```



isset.php

- 2 Now, save the document in your web server's **/htdocs** directory as **isset.php**

3 Next, create an HTML document containing a PHP script to initialize a variable with a submitted value – or with **NULL** if no value has been submitted

```
<?php

if ( isset( $_POST['definition'] ) )
{
    $definition = $_POST['definition'] ;
}
else
{
    $definition = NULL ;
}

# Response statements to be inserted here (Step 4).

?>
```



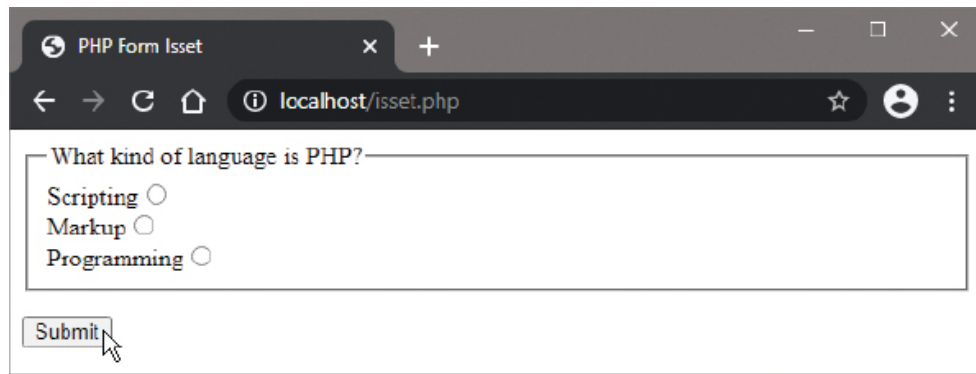
isset_handler.php

4 Finally, insert statements to output an appropriate response according to the value of the variable

```
if ( $definition != NULL )
{
    if ( $definition != 'Scripting' )
    { echo "$definition is Incorrect" ; }
    else
    { echo "$definition is Correct" ; }
}
else
{ echo 'You must select one answer' ; }

?>
```

5 Save the document in your web server's **/htdocs** directory as **isset_handler.php** then open **isset.php** via HTTP and submit the form to see the responses



PHP Form Isset

localhost/isset.php

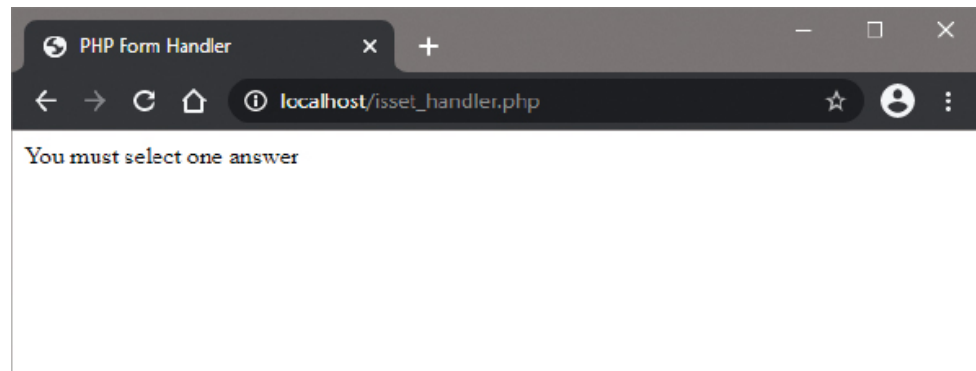
What kind of language is PHP?

Scripting ☒

Markup ☐

Programming ☐

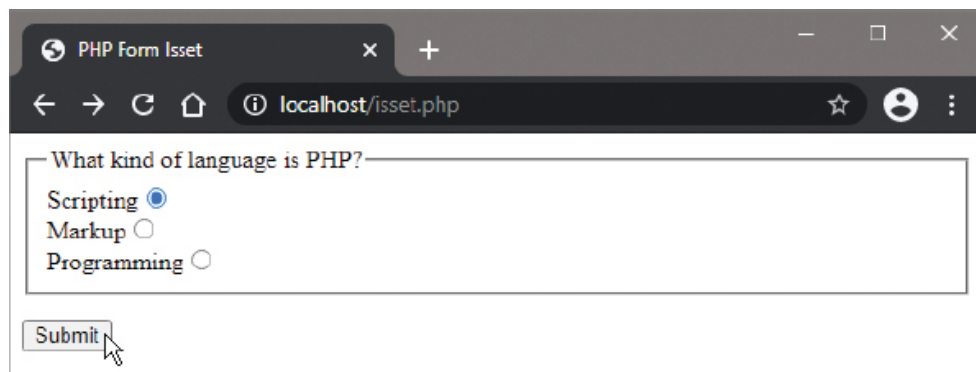
Submit



PHP Form Handler

localhost/isset_handler.php

You must select one answer



PHP Form Isset

localhost/isset.php

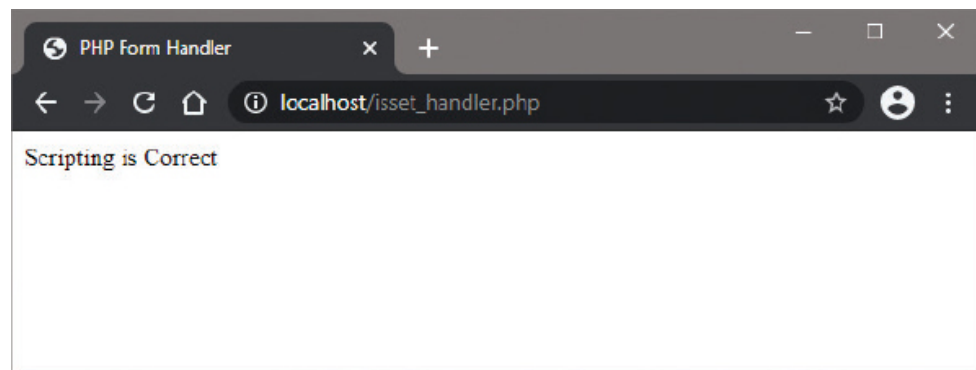
What kind of language is PHP?

Scripting ☒

Markup ☐

Programming ☐

Submit



PHP Form Handler

localhost/isset_handler.php

Scripting is Correct



It is technically more efficient to test whether a condition is not **TRUE** than whether it is **TRUE** – as fewer possibilities need evaluating.

Validating form data

As users may inadvertently submit forms with empty fields, or with data in the wrong format, it is essential your PHP scripts ensure required fields are completed and submitted data is valid.



To ensure the user has entered something into a text field, PHP provides an **empty()** function that accepts a variable argument and returns **TRUE** if its value is an empty string, zero, **NULL** or **FALSE**.

To ensure the user has entered data in numeric format, PHP provides an **is_numeric()** function that accepts a variable argument and returns **TRUE** only if its value is a number.

More specific data validation can be performed using the PHP **preg_match()** function that accepts a “regular expression” argument and a variable argument in which to seek a pattern. Typically, this is used to ensure an email address is in the expected format:

1 Create an HTML document containing a form with two text fields for data submission

```
<form action="valid_handler.php" method="POST">
<fieldset>
<legend>Enter a quantity and email address</legend>
<p>Quantity : <input type="text" name="quantity"></p>
<p>Email Address : <input type="text" name="email"></p>
</fieldset><p><input type="submit" ></p>
</form>
```



valid.php

2 Now, save the document in your web server's **/htdocs** directory as **valid.php**

3 Next, create an HTML document containing a PHP script to initialize a variable with a submitted value – or with **NULL** if no value has been submitted

```
<?php
if ( !empty ( $_POST['quantity'] ) )
{
    $quantity = $_POST['quantity'] ;
    # Format validation to be inserted here (Step 4).
}
else
{ $quantity = NULL ; echo 'You must enter a quantity<br>' ; }
```



valid_handler.php

4 Insert statements to ensure the format is numeric

```
if ( !is_numeric( $quantity ) )
{ $quantity = NULL ; echo 'Quantity must be numeric<br>' ; }
```

5 Now, add statements to initialize another variable with a submitted value – or with **NULL** if no value is submitted

```
if ( !empty ( $_POST['email'] ) )
{
    $email = $_POST['email'] ;
    # Format validation to be inserted here (Step 6).
}
else
{ $email = NULL ; echo 'You must enter an email address' ; }
```

6 Next, insert statements to ensure the email address uses the expected pattern

```
$pattern = '^b[\w.-]+@[ \w.-]+\.[A-Za-z]{2,6}\b' ;
if ( !preg_match( $pattern , $email ) )
{ $email = NULL ; echo 'Email address is incorrect format' ; }
```

7 Finally, add a statement to output valid submitted data

```
if ( ( $quantity != NULL ) && ( $email != NULL ) )  
{ echo "Email : $email<br>Quantity : $quantity " ; }  
?>
```

8 Save the document in your web server's **/htdocs** directory **valid_handler.php** then open **valid.php** via HTTP and submit the form to see the responses

The top screenshot shows a web browser window titled "PHP Form Validation" with the address bar showing "localhost/valid.php". The page content includes a heading "Enter a quantity and email address", two input fields labeled "Quantity :" and "Email Address :", and a "Submit" button. The "Quantity" field contains the text "three" and the "Email Address" field contains "mike@example".

The bottom screenshot shows a web browser window titled "PHP Form Handler" with the address bar showing "localhost/valid_handler.php". The page content displays two lines of text: "Quantity must be numeric" and "Email address is incorrect format".



Strictly speaking, the topic of regular expressions is outside the remit of this book, but you can precisely copy the pattern in this example to ensure email addresses are in the expected format.

Filtering data



To validate user input easily, PHP provides a **filter_var()** function that can be used to both validate and “sanitize” input values. This function requires two arguments to specify the variable to be filtered and a constant name of the filter to be used:

Filter Constant Name	Convention
FILTER_VALIDATE_BOOLEAN	Boolean value
FILTER_VALIDATE_EMAIL	Email address format
FILTER_VALIDATE_FLOAT	Floating-point number
FILTER_VALIDATE_INT	Integer number
FILTER_VALIDATE_IP	IP address format
FILTER_VALIDATE_MAC	MAC address format
FILTER_VALIDATE_REGEXP	Regular expression format
FILTER_VALIDATE_URL	URL format
FILTER_SANITIZE_EMAIL	Strip invalid characters
FILTER_SANITIZE_ENCODED	URL-encode string
FILTER_SANITIZE_MAGIC_QUOTES	Backslash-escape characters
FILTER_SANITIZE_NUMBER_FLOAT	Strip all non-digits
FILTER_SANITIZE_NUMBER_INT	Strip all non-digits
FILTER_SANITIZE_SPECIAL_CHARS	HTML-escape ‘”<>&
FILTER_SANITIZE_STRING	Strip HTML tags

FILTER_SANITIZE_URL	Strip invalid characters
FILTER_CALLBACK	Call a custom filter function



The validation filters return **TRUE** when validation succeeds, otherwise they will return **FALSE**.



Some filters allow further options to be specified. For example, a numeric range can be specified to **FILTER_VALIDATE_INT**.

- 1 Create an HTML document containing a PHP script that begins by writing a styled HTML page heading

```
<?php
$hdr = '<h1 style="color:red">PHP in easy steps</h1>' ;
echo $hdr ;
```



filter.php

- 2 Next, sanitize the page heading by stripping the HTML tags to remove styling and display the stripped version

```
$hdr = filter_var( $hdr , FILTER_SANITIZE_STRING ) ;
echo "Sanitized heading : $hdr" ;
```

- 3 Now, add a function to validate a parameter as being a correctly formatted email address

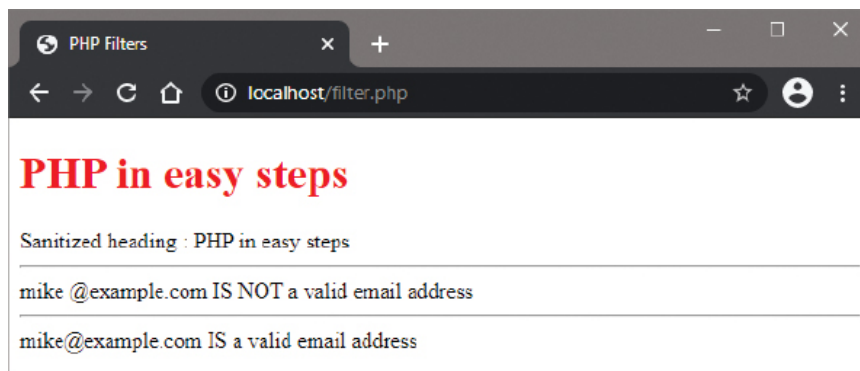
```
function validate( $email )
{
    if ( filter_var( $email , FILTER_VALIDATE_EMAIL ) )
```

```
{ echo( "<hr>$email IS a valid email address" ); }
else
{ echo( "<hr>$email IS NOT a valid email address" ); }
}
```

4 Create an email address containing an illegal space and attempt validation, then sanitize the email address and attempt validation once more

```
$email = 'mike @example.com' ;
validate( $email ) ;
$email = filter_var( $email , FILTER_SANITIZE_EMAIL ) ;
validate( $email ) ;
?>
```

5 Save the document in your web server's **/htdocs** directory as **filter.php** then open it via HTTP to see filtration



Submitted user input can break your webpage so should always be validated.

Sending hidden data

In addition to data entered by the user into HTML form input fields, data generated by PHP can be submitted using HTML hidden form fields. Like regular input fields these also send a name=value pair to the form handler script, but here the value is supplied by PHP, not by the user.



Typically, data submitted using hidden form fields might include the login name of the user and a timestamp of the submission:

- 1 Create an HTML document containing a PHP script to define the timezone and initialize two variables

```
<?php
date_default_timezone_set( 'UTC' );
$time = date( ' H:i , F j ' );
$user = 'Mike' ;
```



hidden.php

- 2 Now, add a statement to write a complete form containing a regular input field and two hidden fields, whose values are set by the variables

```
echo '
<form action="hidden_handler.php" method="POST">
<fieldset>
<legend>Send us your comments</legend>
<textarea rows="5" cols="20" name="comment">
</textarea>
<input type="hidden" name="user" value=" ' . $user . ' ">
<input type="hidden" name="time" value=" ' . $time . ' ">
```

```
</fieldset><p><input type="submit" ></p></form> ‘ ;
?>
```

3 Save the document in your web server’s **/htdocs** directory as **hidden.php**

4 Next, create an HTML document containing a PHP script to initialize a variable with a submitted value – or with **NULL** if no value has been submitted

```
<?php
if ( !empty ( $_POST['comment'] ) )
{
    $comment = $_POST['comment'] ;
}
else
{
    $comment = NULL ; echo ‘You must enter a comment’ ;
}
```



hidden_handler.php



The timestamp in this example is created when the form gets created, not when the form gets submitted.

5 Now, add statements to initialize two variables if the hidden form field values have been set

```
$time =
( !isset ( $_POST['time'] ) ) ? NULL : $_POST['time'] ;

$user =
( !isset ( $_POST['user'] ) ) ? NULL : $_POST['user'] ;
```

6 Finally, add a statement to output valid submitted data

```
if ( ( $comment != NULL ) &&
```

```

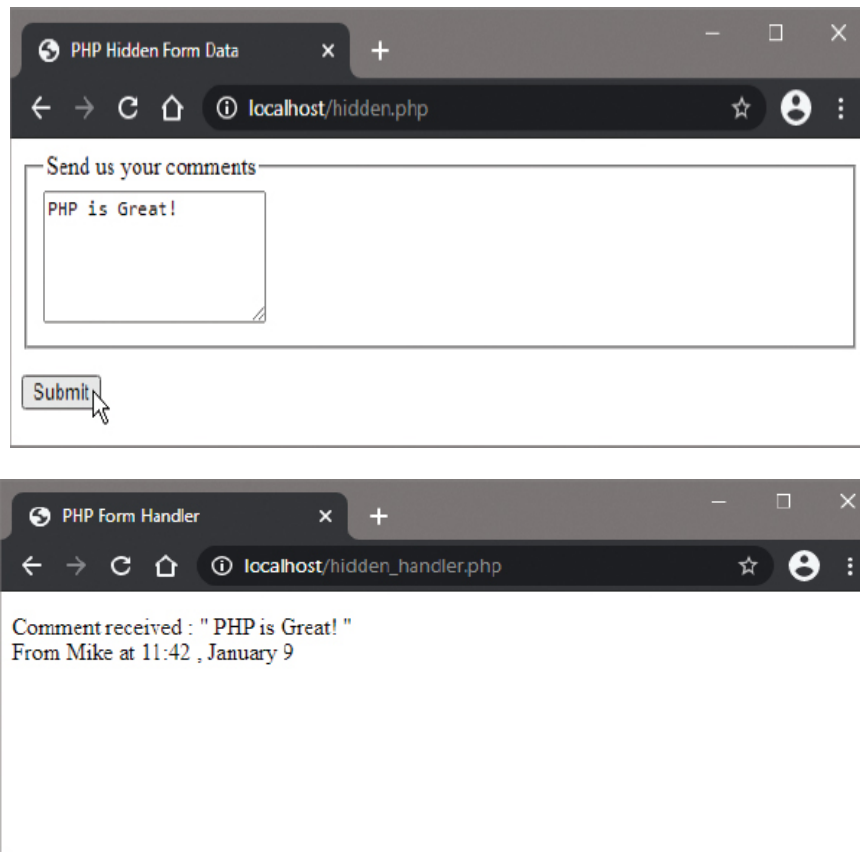
( $time != NULL ) && ( $user != NULL ) )

{ echo "<p>Comment received : \" $comment \"<br>
      From $user at $time </p>\" ; }

?>

```

7 Save the document in your web server's **/htdocs** directory as **hidden_handler.php** then open **hidden.php** via HTTP and submit the form to see the responses



The first screenshot shows a web browser window titled "PHP Hidden Form Data" with the address bar at "localhost/hidden.php". The page contains a form titled "Send us your comments" with a text area containing "PHP is Great!" and a "Submit" button. The second screenshot shows a web browser window titled "PHP Form Handler" with the address bar at "localhost/hidden_handler.php". The page displays the output of the form submission: "Comment received : \" PHP is Great! \"" and "From Mike at 11:42 , January 9".



Notice the ternary **?** : operator is used here in place of a more lengthy **if else** statement.



You can include quotes in output by prefixing them with a backslash to escape them from recognition – as demonstrated here.

Handling submissions

The examples so far in this chapter have each used two documents – one to display an HTML form and another to handle the form submission. This procedure can, however, be performed by a single document, using a simple technique to determine whether a form should be displayed or its submission should be handled.



PHP provides a special “superglobal” `$_SERVER` array variable with a **REQUEST_METHOD** element in which it stores the type of request. Where a form uses the **POST** method for submission, two different types of request are made to the PHP script when the page loads – simply displaying the form uses the standard **GET** method made by most web pages, whereas submitting the form uses the **POST** method. Therefore, the request method can be examined to determine which course of action to take.

Using this technique, the name of the same single page is also specified as the form handler to the HTML **action** attribute. The form can be displayed by the usual **echo** instruction, and validation checks can be made for the response in the same way as described for separate form handler scripts:

- 1 Create an HTML document containing a PHP script that examines the page request method

```
<?php
if ( $_SERVER[ 'REQUEST_METHOD' ] != 'POST' )
{
    # Form display statements to be inserted here (Step 2).
}
else
{
    # Form handling statements to be inserted here (Step 3).
}
```



single.php

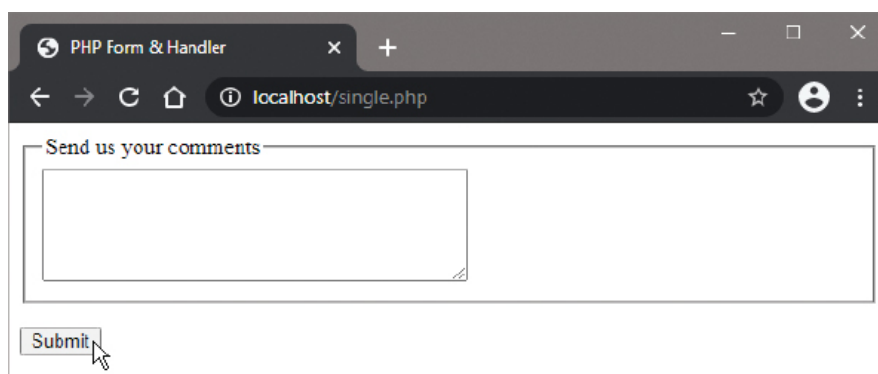
2 Now, insert a statement to display a form when the page has been requested by the standard **GET** method

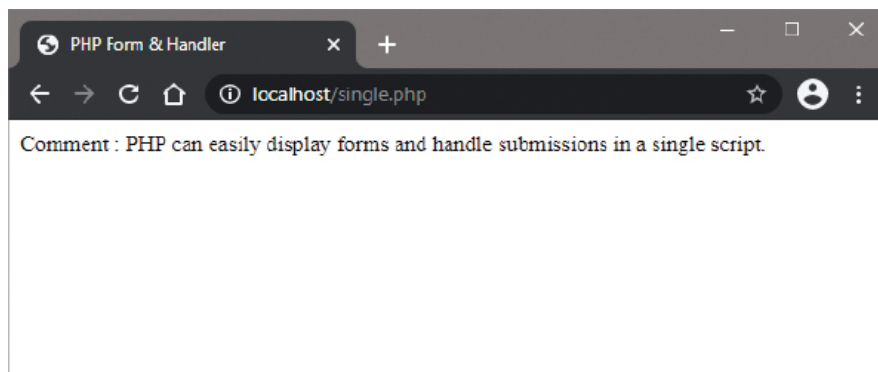
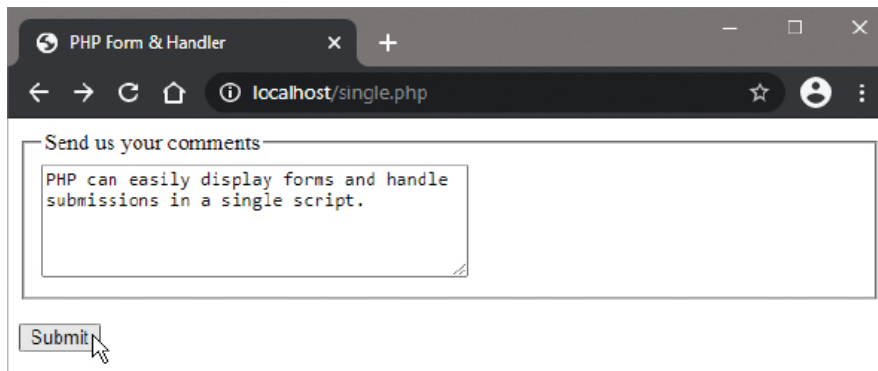
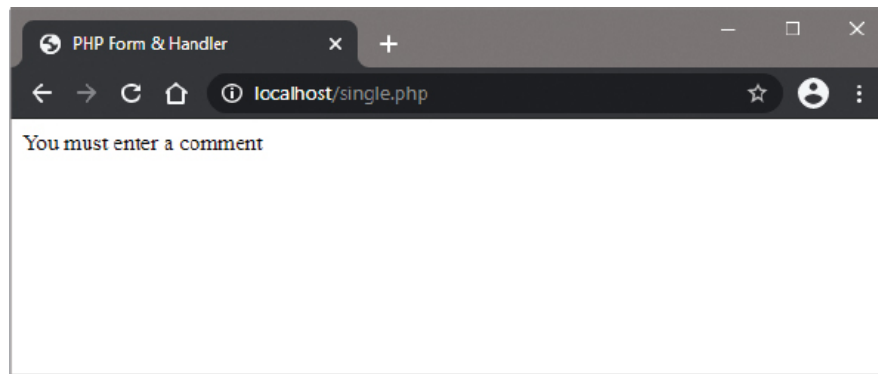
```
echo ‘
<form action=”single.php” method=”POST”>
<fieldset>
<legend>Send us your comments</legend>
<textarea rows=”5” cols=”40” name=”comment”>
</textarea>
</fieldset>
<p><input type=”submit” ></p>
</form> ‘ ;
```

3 Next, insert a statement to handle the form submission when the page has been requested by the **POST** method

```
if ( !empty ( $_POST[‘comment’] ) )
{
    $comment = $_POST[‘comment’] ;
    echo “Comment : $comment “ ;
}
else
{ $comment = NULL ; echo ‘You must enter a comment’ ; }
?>
```

4 Save the document in your web server’s **/htdocs** directory as **single.php** then submit the form to see the responses





The form will be handled by the same page on submission if the action attribute specifies an empty string **action=""** – but it is preferable to specify the page name.

Making sticky forms

When a user attempts to submit a form where a required field has not been completed, it is desirable to retain the entries in the fields they have completed, by making the form “sticky”. This is especially so for lengthy forms with many fields – so the user need not complete all fields once more – and is easily achieved with PHP.



Simply assigning the relevant submitted field data from the **\$_POST** array to the HTML tag’s **value** attribute with **echo** retains the entry when the form is displayed again for completion. As a precaution, the statement should use the **isset()** function to check that the field was in fact completed before submission.

An error message for each incomplete field can be assigned to an array to advise of omissions when the form is displayed again, or a message written confirming success when all fields are completed:

- 1 Create an HTML document containing a sticky form with two text fields for name and email address details

```
<form method="POST"
    action="<?php $_SERVER[ 'PHP_SELF' ] ?>" >
<p>Name:
<input type="text" name="name"
    value="<?php if ( isset( $_POST[ 'name' ] ) )
        echo $_POST[ 'name' ] ; ?>"> </p>
<p>Email:
<input type="text" name="email"
    value="<?php if ( isset( $_POST[ 'email' ] ) )
        echo $_POST[ 'email' ] ; ?>"></p>
<p><input type="submit"></p>
</form>
```



sticky.php

2 Now, add a PHP script to handle the form submission

```
<?php

if ( $_SERVER[ 'REQUEST_METHOD' ] == 'POST' )
{
    // Statements to be inserted here (Steps 3-5).
}

?>
```

3 Next, insert into the PHP script block a statement to create an array for error messages

```
$errors = array() ;
```



`$_SERVER[ 'PHP_SELF' ]` is used by this example to ensure the script will still work if renamed.



The `if` statement tests that the form has been submitted by checking the request method is **POST**, not the **GET** method that is used to display the empty form.

4 Then, insert statements to initialize variables with submitted values, or add to the errors array for omissions

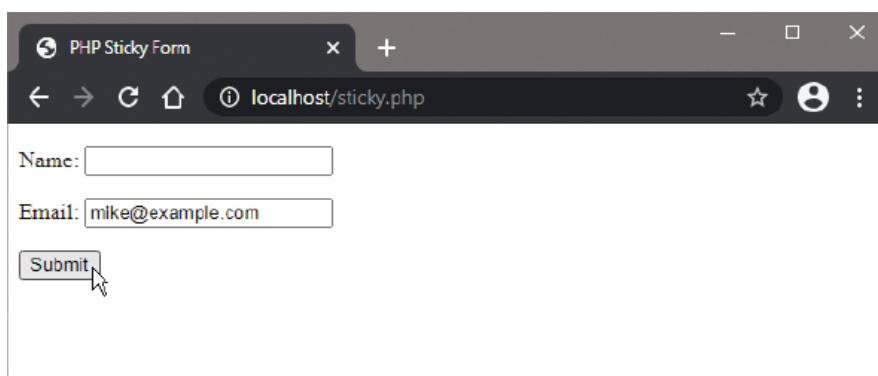
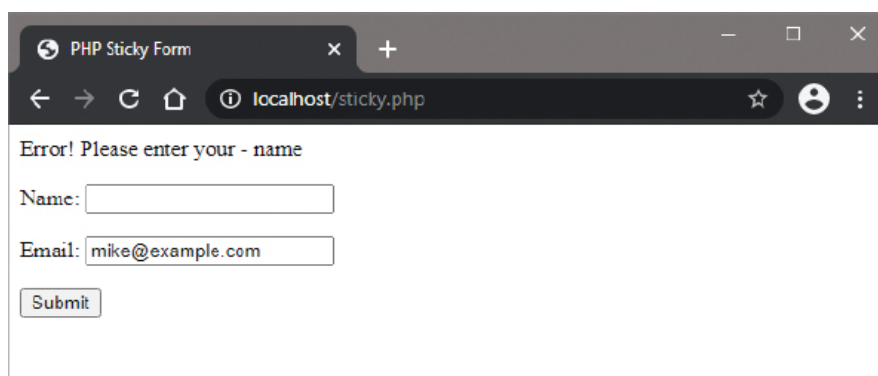
```
if ( empty( $_POST[ 'name' ] ) ) { $errors[] = 'name' ; }
else { $name = trim( $_POST[ 'name' ] ) ; }

if ( empty( $_POST[ 'email' ] ) ) { $errors[] = 'email' ; }
else { $email = trim( $_POST[ 'email' ] ) ; }
```

5 Now, insert statements to write error messages or confirm successful form submission

```
if( !empty( $errors ) )
{ echo 'Error! Please enter your ' ;
  foreach ( $errors as $msg ) { echo " - $msg " ; } }
else { echo "Success! Thanks $name " ; }
```

6 Save the document in your web server's **/htdocs** directory as **sticky.php** then open it via HTTP and submit the form to see the retained values and response messages


It is good practice to use the **trim()** function, as seen here, to remove any leading and trailing spaces from the submitted field value.

Uploading files



The PHP **\$_FILES** superglobal variable makes it simple to **POST** file uploads from an HTML form if the **<form>** tag includes an **enctype** attribute set to “multipart/formdata”. When submitted, a **\$_FILES** associative array is created with keys that include the file **name**, a generated temporary name (**tmp_name**), and the file **size**. The filename and size can be used for validation. The temporary filename can then be specified to the **move_uploaded_file()** function along with the original filename to upload the file:

1 Create an HTML document containing a sticky form with a file type input named “image”

```
<form action="<?php $_SERVER[ 'PHP_SELF' ] ?>"
      method="POST" enctype = "multipart/formdata" >
```

Select an image to upload :

```
<input type="file" name="image" >
<input type="submit" value="Upload Image"> </form>
```



upload.php

2 Now, add a PHP script to handle the form submission

```
<?php

if ( $_SERVER[ 'REQUEST_METHOD' ] == 'POST' )
{
    // Statements to be inserted here (Steps 3-7).
}

?>
```

3 Next, insert statements to initialize three variables with information about the file selected by the user

```
$name = $_FILES[ 'image' ][ 'name' ] ;
$tmp_name = $_FILES[ 'image' ][ 'tmp_name' ] ;
$size = $_FILES[ 'image' ][ 'size' ] ;
```

4 Now, insert statements to ensure the selected file is of a desired file type by testing the file extension, or quit

```
$ext = pathinfo( $name , PATHINFO_EXTENSION ) ;
$ext = strtolower( $ext ) ;
if( $ext != 'png' && $ext != 'jpg' && $ext != 'gif' )
{ echo 'Format must be PNG, JPG, or GIF' ; exit() ; }
```

5 Insert a statement to ensure the selected file is of a desired file size, or quit

```
if( $size > 512000 )
{ echo 'File size must not exceed 500Kb' ; exit() ; }
```



The PHP configuration must permit file uploads for this example to work. The configuration file **php.ini**, within the PHP directory, must contain a line **file_uploads = On**.



Notice how the **pathinfo()** function provides the file extension part of the filename here.

6 Insert a statement to ensure the selected file does not have the same name as one already on the server, or quit

```
if( file_exists( $name ) )
{ echo 'File ' . $name . ' already uploaded' ; exit() ; }
```

7 Finally, attempt to upload the file, naming it by its original filename and displaying it below the form

```
try
```

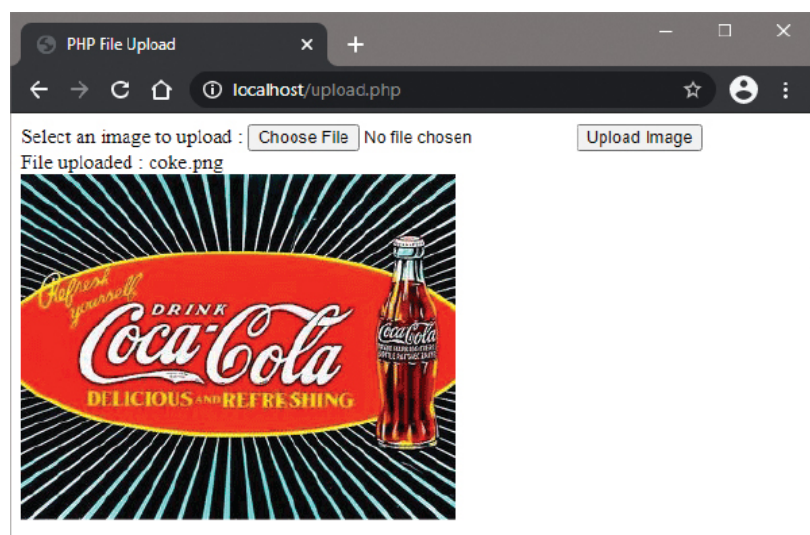
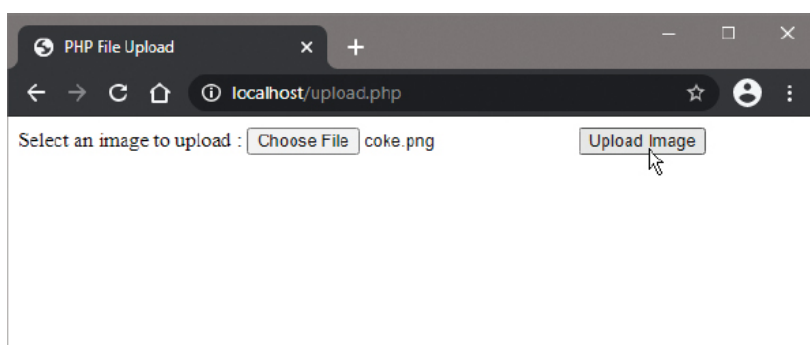


```

{
    move_uploaded_file( $temp , $name );
    echo 'File uploaded : '.$name ;
    echo '<br>' ;
}
catch( Exception $e )
{
    echo 'File upload failed!' ;
}

```

8 Save the document in your web server's **/htdocs** directory as **upload.php** then open it via HTTP and select a suitable image file for upload to the **/htdocs** directory



coke.png



The arguments to the **move_uploaded_file()** function specify the (temporary) filename and destination. It could usefully specify a dedicated directory path destination, such as **‘uploads/’.\$name**.

Surrounding forms

PHP usefully allows multiple files to be incorporated together to create a single web page. This means that dynamic forms can be surrounded by static header and footer content that can be easily reused to create other pages within a website.



The PHP **include()** function takes the URL of another file to be incorporated within a web page as its argument. Typically, included static content files are HTML and are located in an “includes” folder within the **/htdocs** directory of your web server.

PHP also provides a **require()** function that works just like the **include()** function to incorporate other files within a web page, but is subtly different. When the **include()** function fails, as the specified file cannot be found, for example, a warning gets sent to the browser but the script continues to run. When the **require()** function fails, an error is printed but the script gets halted. Usually, the **include()** function incorporates static HTML files, whereas the **require()** function incorporates dynamic PHP scripts.

Importantly, the title of a web page incorporating multiple files can be dynamically written by PHP to allow maximum flexibility of otherwise static HTML header content:

- 1 Create an HTML document containing static header content and a PHP snippet to write the page title

```
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="UTF-8">
<title> <?php echo $page_title ; ?> </title>
<link rel="stylesheet" href="includes/style.css">
</head>

<body>
<header><h1>Page Header</h1></header>
```



header.html

2

Now, create an HTML document containing footer content

```
<footer><p>Page Footer</p></footer>
</body>
</html>
```



footer.html

3

Next, create a folder named “includes” in the **/htdocs** directory of your web server and save the above documents inside it as **header.html** and **footer.html** respectively

4

Create a CSS style sheet to present the header and footer content (and rules to present aspects of later examples)

```
header > h1 { border-bottom : 1px dashed black ;
               font-style : italic ; font-size : x-large ; }
footer > p { border-top : 1px dashed black ;
             font-style : italic ; }
table { border-spacing : 5px ; width : 530px ; }
th { color : #FFF ; background : #000 ; text-align : left ; }
td { border-bottom : 1px solid black ;
      padding : 3px ; background : #F0F0F0 ;
      text-align : left ; vertical-align : top ; }
p#err_msg { color : #F00 ; font-weight : bold ; }
```



style.css

5

Save the style sheet in the **/includes** folder you have created, alongside the HTML header and footer files

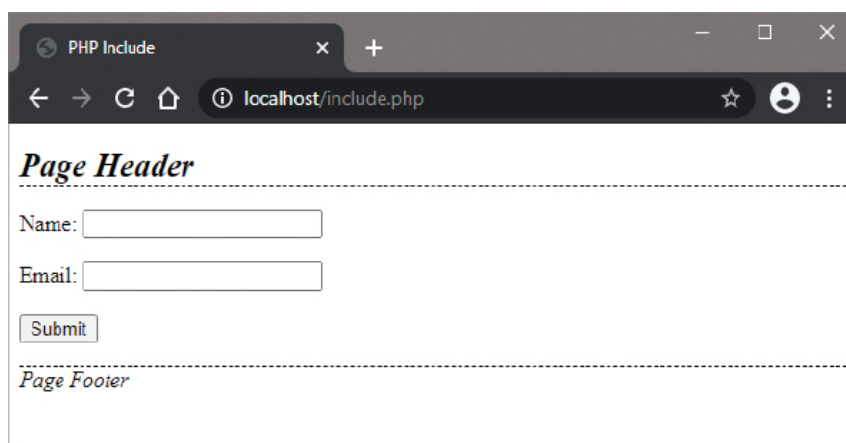
6 Finally, create a PHP script that sets the web page title and surrounds a form with header and footer content

```
<?php
$page_title = 'PHP Include' ;
include ( 'includes/header.html' ) ;
echo '<form action="include.php" method="POST">
<p>Name: <input type="text" name="name"> </p>
<p>Email: <input type="text" name="email" ></p>
<p><input type="submit"></p></form>' ;
include ( 'includes/footer.html' ) ;
?>
```



include.php

7 Save the PHP script in your web server's **/htdocs** directory as **include.php** then open it via HTTP to see the page title set and a form surrounded by static content



You can use your browser's View Source facility to see that the entire content has been effectively incorporated into a single web page.

Appending link data

PHP predefined “superglobal” variables, such as the **\$_SERVER** array variable that stores the page request method and the **\$_SESSION** array variable that stores user data across pages of a website until the user quits, are accessible in all scopes of any script.



Where an HTML form submits data using the **POST** method, the **\$_POST** array stores the value of submitted form values in like-named array elements. So, the value of a submitted field named “city” can be accessed using **\$_POST['city']**.

Similarly, where data is submitted using the **GET** method, the **\$_GET** array stores the value of submitted values in named array elements. So the value of a submitted data named “id” can be accessed using **\$_GET['id']**. This is typically used to submit data to a script by appending a name=value pair to the URL in an HTML hyperlink anchor, like this:

```
<a href="www.example.com/script.php?id=1">
```

The script receiving the data, nominated by the hyperlink, may then use the passed value to determine a response:

- 1 Create an HTML document containing a PHP script that seeks an element named “id” within the **\$_GET** array

```
<?php
if ( isset ( $_GET[ 'id' ] ) )
{
    # Handler statements to be inserted here (Steps 2-3).
}
```



link.php

3 Next, insert a handler statement to assign a passed value to a like-named PHP variable

```
$id = $_GET[ 'id' ] ;
```

4 Now, insert a handler statement to output an appropriate response according to the value passed

```
switch( $id )
{
    case 1 : echo 'Cow selected<hr>' ; break ;
    case 2 : echo 'Dog selected<hr>' ; break ;
    case 3 : echo 'Goat selected<hr>' ; break ;
}
```

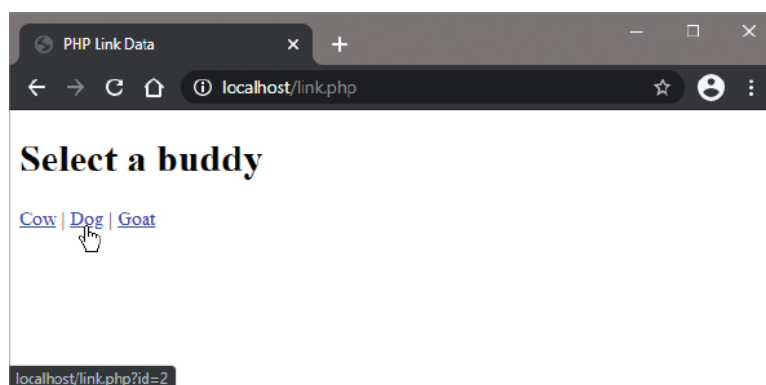


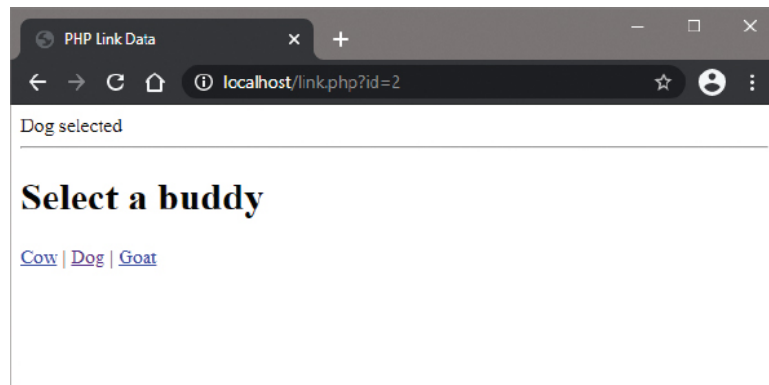
The `$_SESSION` superglobal will be used to store user details in Chapter 10.

4 Finally, add statements to display hyperlinks with appended values

```
echo '<h1>Select a buddy</h1>' ;
echo ' <p><a href="link.php?id=1">Cow</a> | ' ;
echo ' <a href="link.php?id=2">Dog</a> | ' ;
echo ' <a href="link.php?id=3">Goat</a></p>' ;
?>
```

5 Save the PHP script in your web server's `/htdocs` directory as `link.php` then open it via HTTP and click the links to see the responses for the passed values





PHP also provides a `$_REQUEST` superglobal that contains the entire contents of the `$_POST` and `$_GET` arrays. This means that `$_POST['city']` is also accessible using `$_REQUEST['city']` but this should be avoided. Always use the specific array for the type of submission instead.



Notice that the passed name=value pair is visible in the browser address field.

Summary

- An HTML **<form>** tag's **action** attribute specifies the PHP script to handle the form submission, and its **method** attribute specifies how the data is sent to the script.
- Form data submitted by the **POST** method gets automatically stored in the PHP superglobal **\$_POST** array variable.
- Elements of the **\$_POST** array are named the same as the HTML form element containing each submitted value.
- PHP can ensure that the user has entered data into submitted HTML form fields using the built-in **isset()** function.
- The PHP **empty()** function checks whether a value has been entered into a form text field.
- Format validation can be performed on submitted form data by the **is_numeric()** function and **preg_match()** function.
- Data generated by PHP, such as a timestamp created by the **date()** function, can be submitted from hidden form fields.
- The PHP superglobal **\$_SERVER['REQUEST_METHOD']** array element stores the type of request made to load the page.
- Sticky forms retain their field data simply by writing any submitted values back to the HTML input's **value** attribute.
- The PHP **include()** and **require()** functions are used to incorporate other content and scripts within a web page.
- Data submitted by the **GET** method gets automatically stored in the PHP superglobal **\$_GET** array variable.
- Hyperlinks can have name=value pairs appended to submit data to a script using the **GET** method.